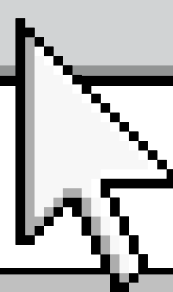


DATA MINING E BUSCA HEURÍSTICA:

UMA APLICAÇÃO DE *WEB SCRAPING* E BUSCA
META-HEURÍSTICA

Fatec
Baixada Santista
Rubens Lara

Iniciar

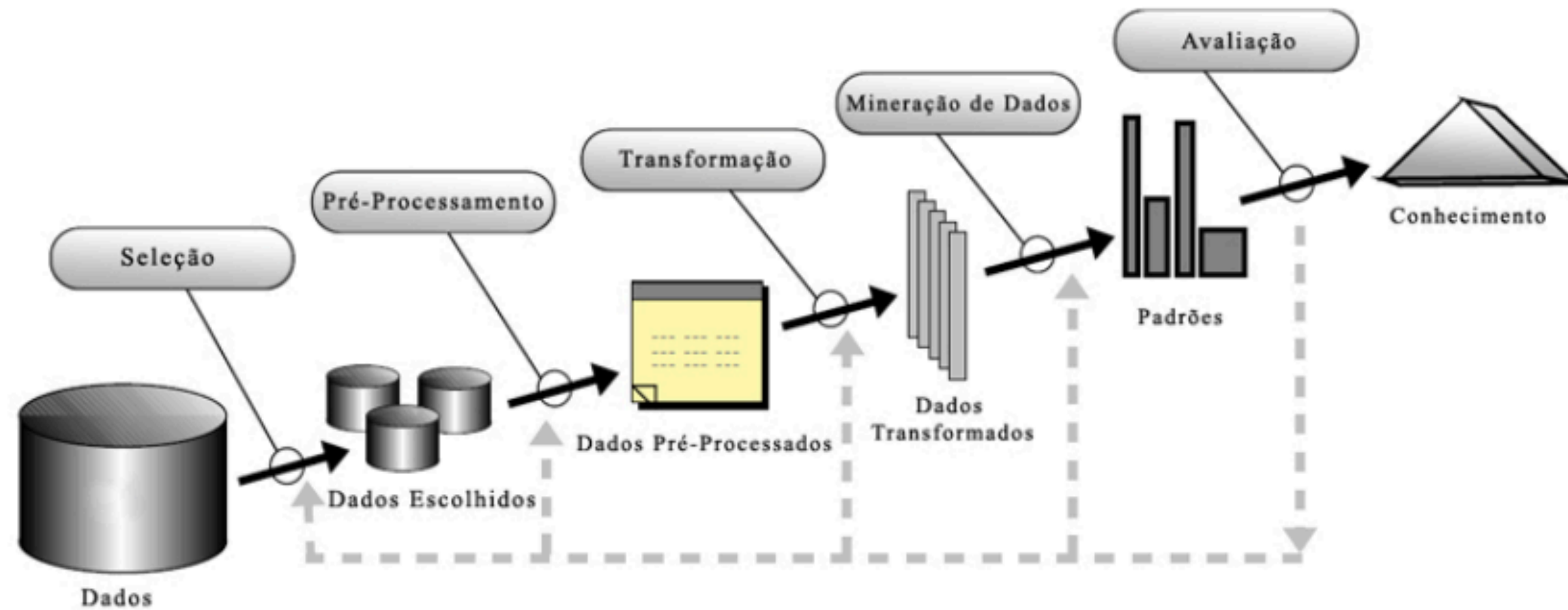




DEFINIÇÃO

data mining é uma etapa fundamental no processo de descoberta de conhecimento em bancos de dados (KDD) consiste na aplicação de algoritmos e na análise de dados a fim de encontrar padrões que auxiliem na tomada de decisões





Ciclo do KDD (knowledge discovery in databases)





TAREFAS

MÉTODOS

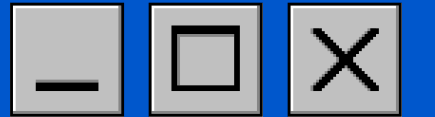
Classificação → SVM, redes neurais, árvores de decisão, algoritmo genético

Regressão/Predição → Regressão linear e não-linear

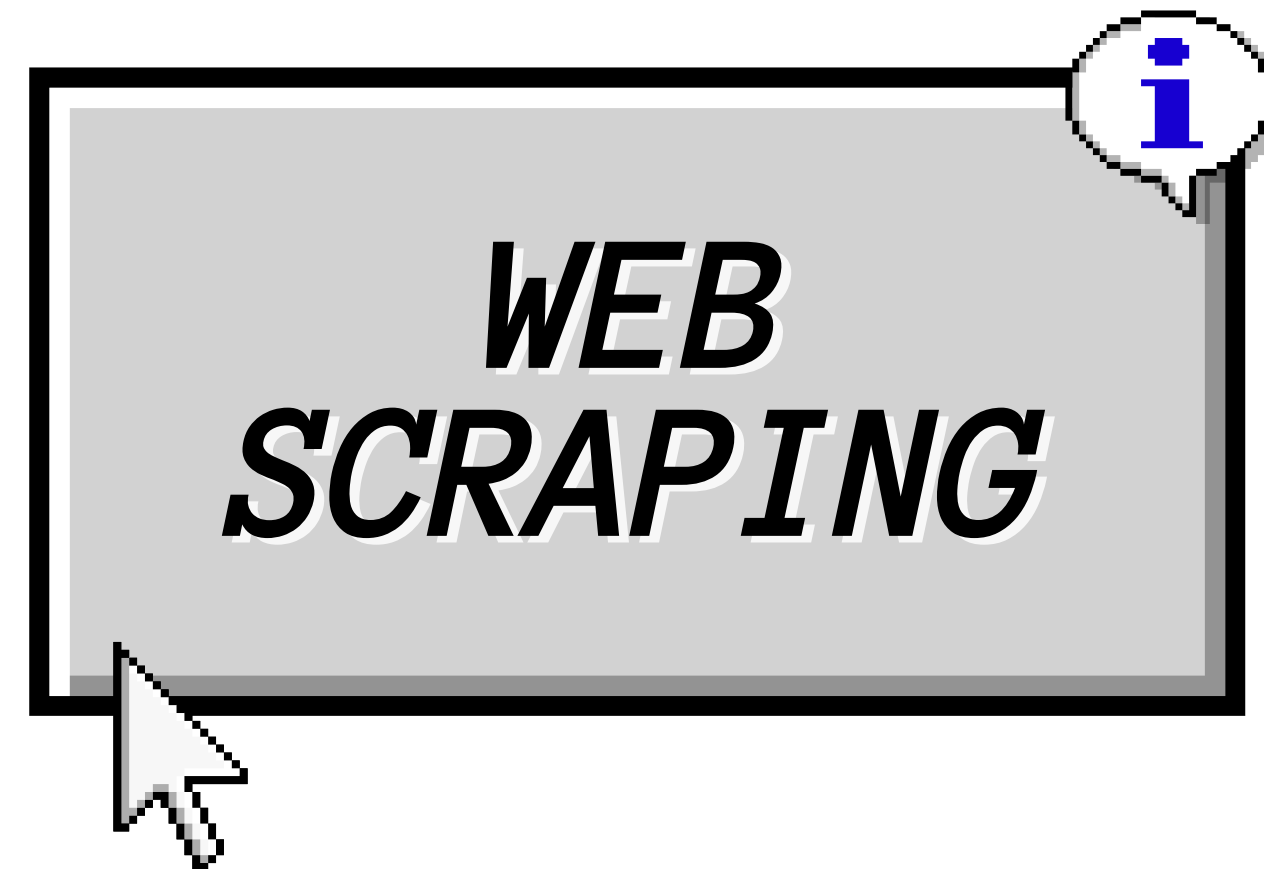
Agrupamento → K-means, K-meloids, métodos baseados em grade

Associação → Mineração de itens frequentes

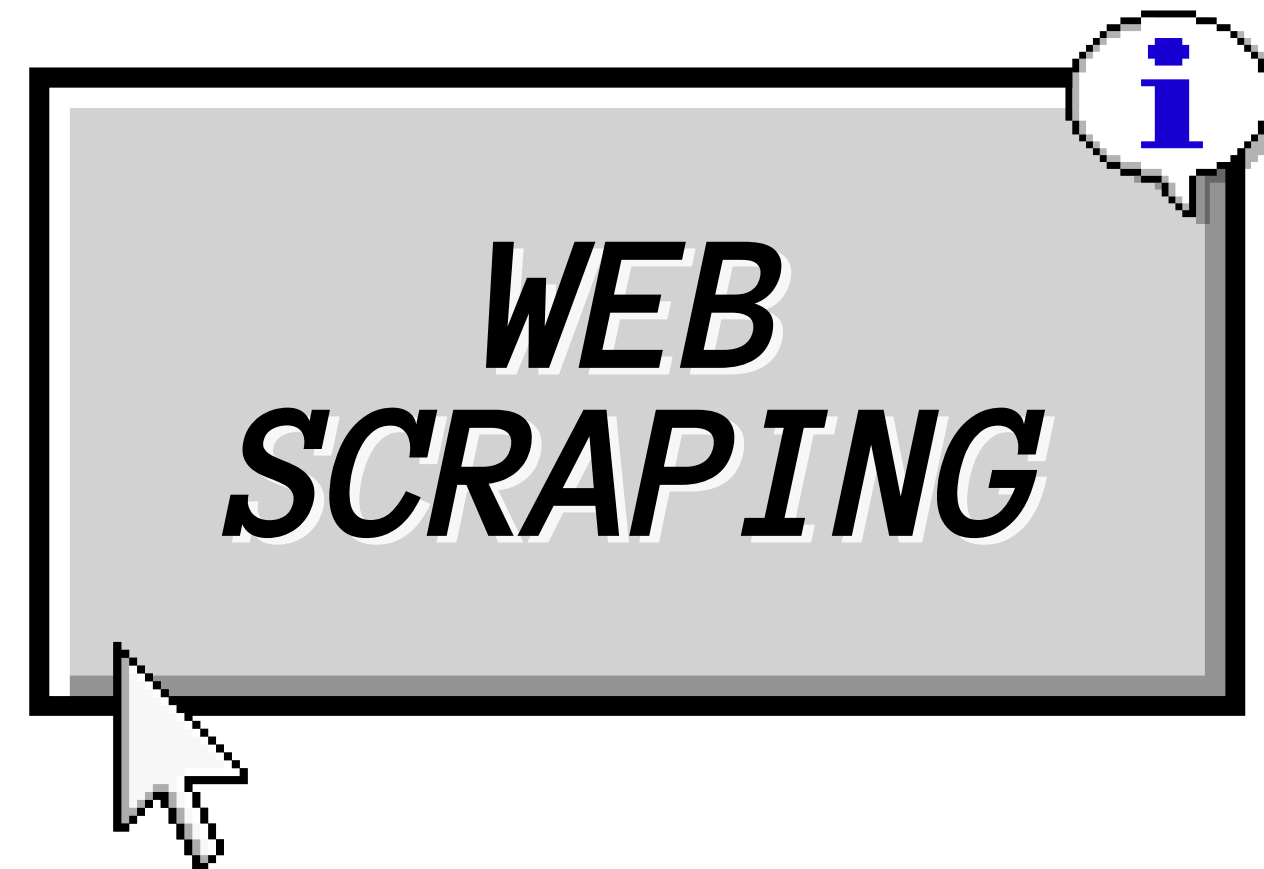


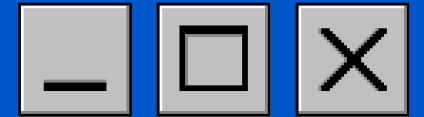


A raspagem de dados é comumente associadas à mineração na etapa de extração, as ferramentas são poderosas juntas porque a raspagem permite, já na extração, tratar, padronizar e eliminar dados gerando uma base estruturada que pode ser submetida a outras tarefas. Então a raspagem não apenas extrai como aumenta a qualidade da coleção e estrutura os dados.



Considerando isso, para a aplicação de mineração de dados, aplicou-se dois algoritmos de raspagem de dados na página de *games* Steam para obter avaliações referente ao jogo Brawlhalla, desenvolvendo a tarefa de extração e padronização para obter um subconjunto das avaliações em formato estruturado que pudesse ser utilizado na aplicação do outro tema.





EXTRAÇÃO DOS DADOS

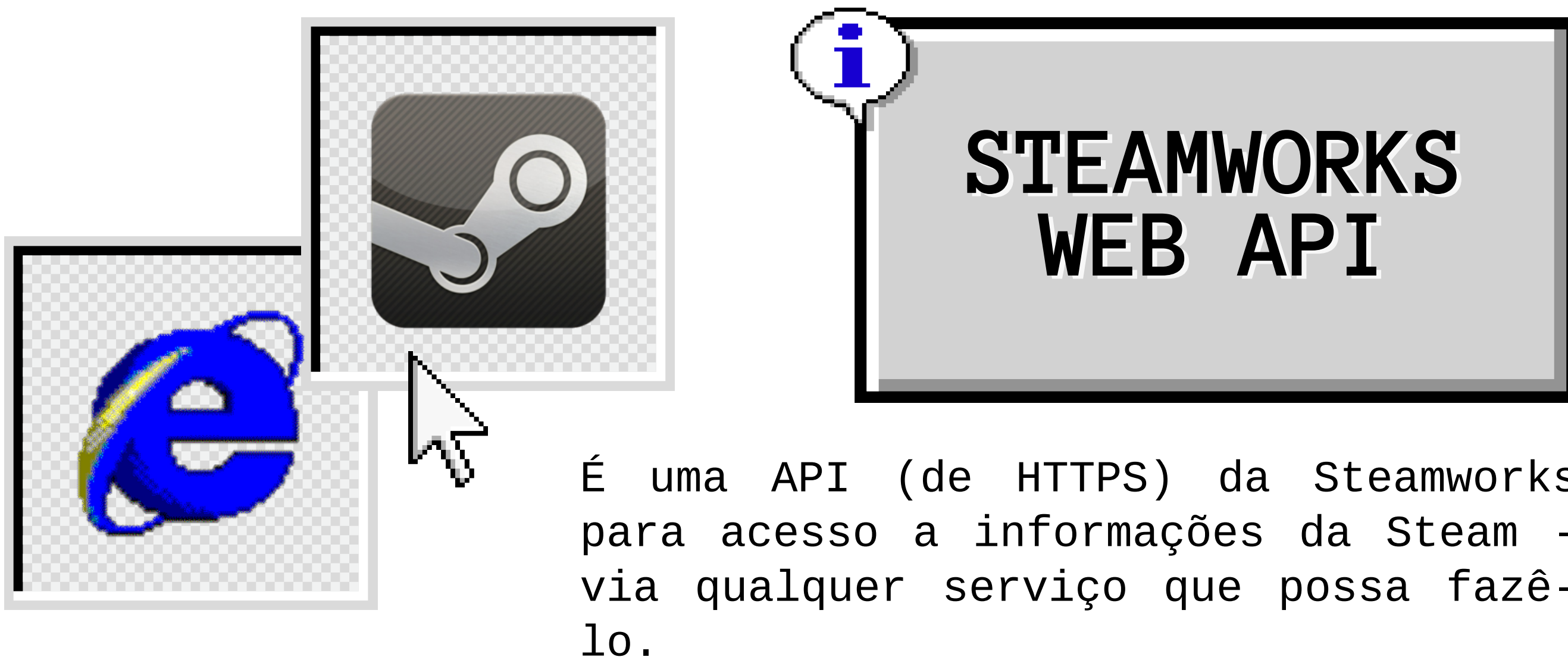
Escolhemos a **Steam** como fonte de dados, isso foi feito pois: há uma facilidade de acesso e coleta dos dados na plataforma, majoritariamente devido ao fato de a Steam ser bem aberta para Scraping de todo tipo.

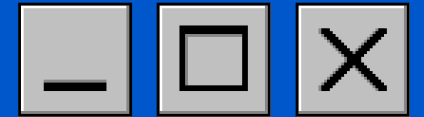


FORMAS DE EXTRAÇÃO

1. A primeira forma de Data Mining foi feita com auxílio da biblioteca **BeautifulSoup**. Ela extrai diretamente o código HTML da página. Mais de 10.000 reviews em português foram extraídas assim. Contudo, era mais complexo para extrair certos dados e também não era aberta para todos os jogos.
2. A segunda forma foi por meio da **API da SteamWorks**, sendo essa, por sua facilidade e alto índice de controle a forma canônica de extração de dados.







REQUISIÇÕES DA API

A estrutura de uma requisição pela API segue a seguinte estrutura:

<https://api.steampowered.com/<interface>/<method>/v<version>/>

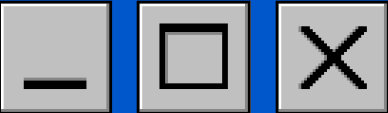
Para se pegar reviews, a estrutura básica é a seguinte:

[GET](#) store.steampowered.com/appreviews/<appid>?json=1

Exemplo de uma requisição complexa, do jogo **Stardew Valley**, com comentários ***offtopics*** e em **português**.

[GET](#) "<https://store.steampowered.com/appreviews/3159330?json=1&filter=offtopic&activity=0&language=portuguese>"





DADOS BRUTOS

JSONRaw DataHeaders

SaveCopyCollapse AllExpand AllFilter JSON

success:1

▼ query_summary:

num_reviews:20

review_score:9

review_score_desc:"Overwhelmingly Positive"

total_positive:455107

total_negative:5116

total_reviews:460223

▼ reviews:

▼ 0:

recommendationid:"208409508"

▼ author:

steamid:"76561199363389036"

num_games_owned:0

num_reviews:1

playtime_forever:8020

playtime_last_two_weeks:2990

playtime_at_review:2733

last_played:1763502767

language:"english"

review:"Love this game, but please remember to go to bed before 2 either in the game or in real life."

timestamp_created:1762278506

timestamp_updated:1762278506

voted_up:true

votes_up:193

votes_funny:84

weighted_vote_score:"0.939383566379547119"

comment_count:2

steam_purchase:true

STDIN

1{

2"success": 1,

3"query_summary": {

4"num_reviews": 9,

5"review_score": 9,

6"review_score_desc": "Overwhelmingly Positive",

7"total_positive": 1921,

8"total_negative": 22,

9"total_reviews": 1943

10},

11"reviews": [

12{

13"recommendationid": "206628707",

14"author": {

15"steamid": "76561198324296157",

16"num_games_owned": 50,

17"num_reviews": 18,

18"playtime_forever": 771,

19"playtime_last_two_weeks": 0,

20"playtime_at_review": 602,

21"last_played": 1760533537

22},

23"language": "portuguese",

24"review": "fiquei surpeendido , é bom para pas

25to tudo okay",

26"timestamp_created": 1760358139,

27"timestamp_updated": 1760358139,

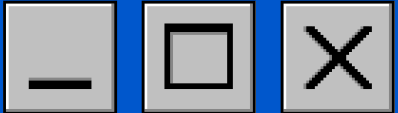
28"voted_up": true,

29"votes_up": 0,

Requisição vista pelo Firefox.

Requisição vista pelo terminal (GNOME).



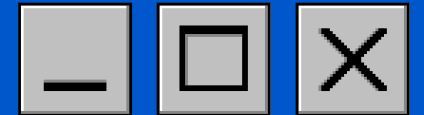


PARÂMETROS

Nome	Tipo	Descrição
filter	string	recent – sorted by creation time updated – sorted by last updated time all – (default) sorted by helpfulness, with sliding windows based on day_range parameter, will always find results to return. If paging through the reviews with cursor then choose either the recent option or the updated option to eventually receive an empty response list.
language	string	see https://partner.steamgames.com/doc/store/localization/languages (and use the API language code list) or pass “all” for all reviews
day_range	string	range from now to n days ago to look for helpful reviews. Only applicable for the “all” filter. Maximum value is 365.
cursor	string	reviews are returned in batches of 20, so pass "*" for the first set, then the value of "cursor" that was returned in the response for the next set, etc. Note that cursor values may contain characters that need to be URLEncoded for use in the querystring.
review_type	string	all – all reviews (default) positive – only positive reviews negative – only negative reviews
purchase_type	string	all – all reviews non_steam_purchase – reviews written by users who did not pay for the product on Steam steam – reviews written by users who paid for the product on Steam (default)
num_per_page	string	by default, up to 20 reviews will be returned. More reviews can be returned based on this parameter (with a maximum of 100 reviews)
filter_offtopic_activity	number	by default, off-topic reviews (aka "Review Bombs") are filtered out and are not returned in this API. Pass 0 to include them. See here .

Fonte: <https://partner.steamgames.com/doc/store/getreviews>





DEFINIÇÕES

As bibliotecas utilizadas foram:

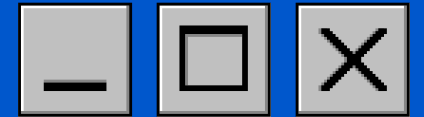
- **request** - requerimentos HTTPs;
- **pandas** - transformar em .CSVs;
- **time** - puramente auxiliar;

Ou seja, não foi necessário uma biblioteca para acessar esa API - já que era uma interface WEB.

```
# Setup ----  
import requests  
import pandas as pd  
import time
```

```
def extrairReviews(params: dict, limite: int = 1e3):  
    """  
    Acesso a API da SteamWorks utilizando uma operação GET.  
  
    params: Dicionário dos parâmetros da função.  
    limite: Número máximo de reviews para serem pegas  
    """  
    reviews = []  
    while len(reviews) < limite:  
        # Simula o GET  
        resp = requests.get(f'https://store.steampowered.com/appreviews/{app_id}', params=params)  
        data = resp.json()  
        # Pega as reviews da variável em json  
        reviews_atual = data.get('reviews')  
  
        if not reviews_atual:  
            break  
  
        reviews.extend(reviews_atual)  
  
        # Move para a próxima página  
        cursor = data.get('cursor')  
        if not cursor or cursor == params['cursor']:  
            break  
  
        params['cursor'] = cursor  
  
        # Print do total de reviews:  
        print("Total extraído: ", len(reviews))  
  
        # Espera para não acharem ser DDOS  
        time.sleep(1)  
  
    # Por agora tá tacando literalmente todas as infos no .csv  
    return pd.DataFrame(reviews)
```





CÓDIGO

```
# Extração ----
app_id = 291550 # Código do Brawhalla

# Dicionário de parâmetros
params = {
    'json': 1,
    'review_type': 'positive',
    # Em inglês tem mais reviews (em geral)
    'language': 'english',
    'filter': 'recent',
    'num_per_page': 100,
    'cursor': '*',
    'filter_offtopic_activity': 0,
    'purchase_type': 'all'
}

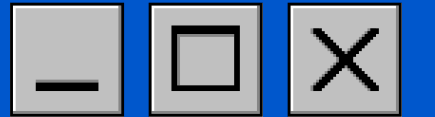
limite = 5200

# Positivos
params["review_type"] = 'positive'

print("Extraindo reviews positivas.")
reviews = extrairReviews(params, limite)
reviews.to_csv('SteamAPI/brawhalla_pos.csv', index=False)
print(f"Reviews positivas coletadas: {len(reviews)}.")
```

```
SteamAPI > brawhalla_pos.csv > data
1 recommendationid,author,language,review,timestamp_created,timestamp_updated,voted
2 208422304,"{'steamid': '76561199055910088', 'num_games_owned': 0, 'num_reviews':
3 208422013,"{'steamid': '76561199861757913', 'num_games_owned': 0, 'num_reviews':
4 208421834,"{'steamid': '76561199538229970', 'num_games_owned': 0, 'num_reviews':
5 208415054,"{'steamid': '76561199021684601', 'num_games_owned': 48, 'num_reviews':
6 208403153,"{'steamid': '76561199055868688', 'num_games_owned': 0, 'num_reviews':
7
8 ohhhhahhh",1762274080,1762274080,True,0,0,0.5,0,False,False,False,False
9 208401929,"{'steamid': '76561199229876550', 'num_games_owned': 0, 'num_reviews':
10 208401658,"{'steamid': '76561199048840852', 'num_games_owned': 70, 'num_reviews':
11 208393118,"{'steamid': '76561199071575490', 'num_games_owned': 0, 'num_reviews':
12 208392879,"{'steamid': '76561199848038637', 'num_games_owned': 0, 'num_reviews':
13 208390753,"{'steamid': '76561198286435697', 'num_games_owned': 0, 'num_reviews':
14 208386293,"{'steamid': '76561199493072655', 'num_games_owned': 18, 'num_reviews':
15 ",1762258763,1762258763,True,0,0,0.5,0,False,False,False,False
16 208385555,"{'steamid': '76561198298731305', 'num_games_owned': 158, 'num_reviews':
17 208377419,"{'steamid': '76561199584291448', 'num_games_owned': 5, 'num_reviews':
18 208371083,"{'steamid': '76561199207970183', 'num_games_owned': 145, 'num_reviews':
19 208368082,"{'steamid': '76561199852892690', 'num_games_owned': 0, 'num_reviews':
20 ",1762231092,1762231092,True,0,0,0.5,0,False,False,False,False
21 208365380,"{'steamid': '76561198853180951', 'num_games_owned': 0, 'num_reviews':
22 208364743,"{'steamid': '76561199172386585', 'num_games_owned': 0, 'num_reviews':
23 208361303,"{'steamid': '76561199239766310', 'num_games_owned': 0, 'num_reviews':
24 208356891,"{'steamid': '76561198791889036', 'num_games_owned': 0, 'num_reviews':
25 208348958,"{'steamid': '76561199584010608', 'num_games_owned': 7, 'num_reviews':
26 ",1762207157,1762207157,True,0,0,0.5,0,False,False,False,False
27 208348074,"{'steamid': '76561199521778431', 'num_games_owned': 0, 'num_reviews':
```



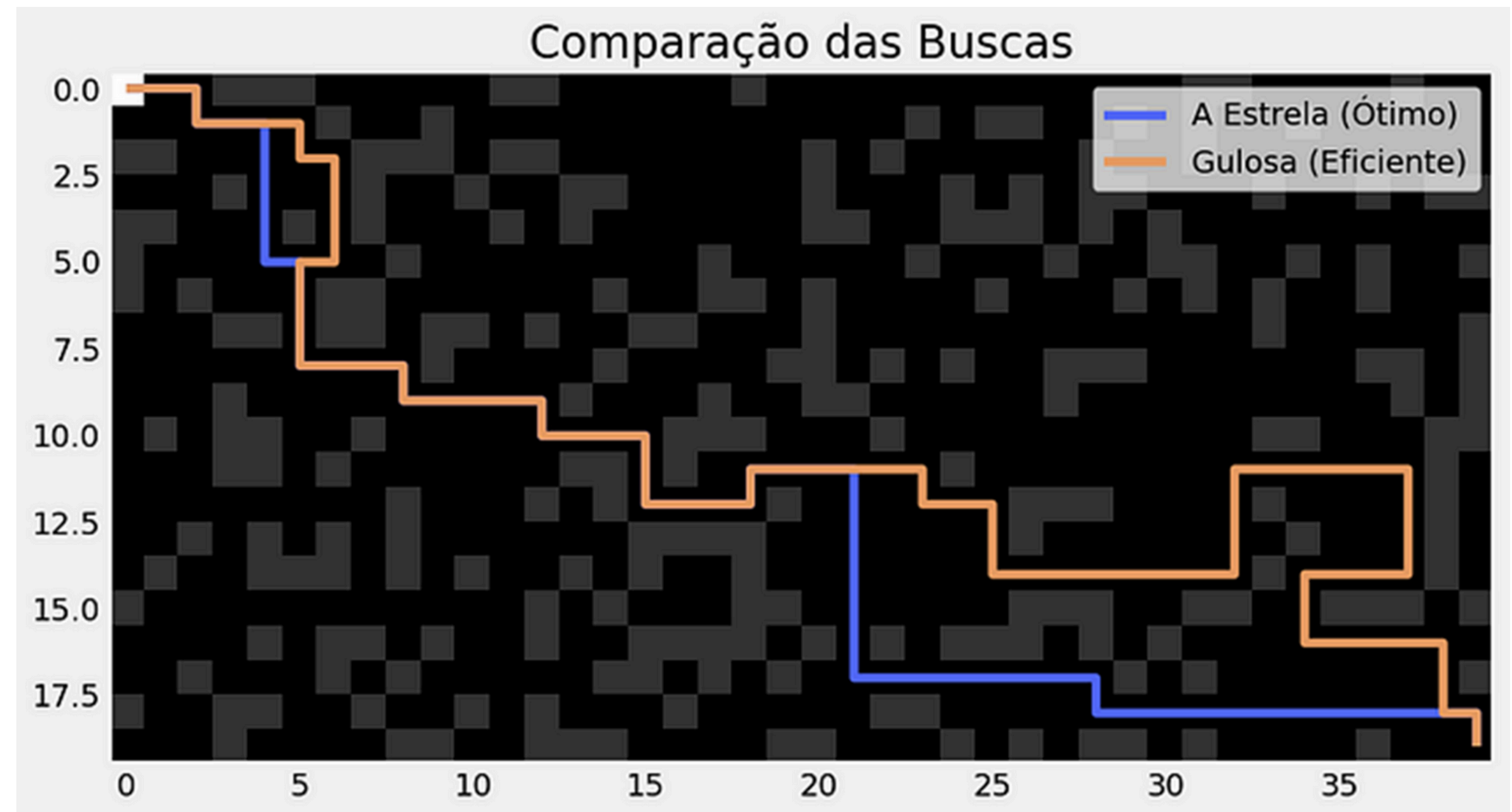


A busca heurística é uma técnica de otimização para solucionar problemas complexos, especialmente quando há grande expansão de estados.

Quando não é possível explorar todas as alternativas, ela se torna uma boa opção se baseando na aproximação contínua e progressiva do problema apresentado.



BUŞCA HEURÍSTICA





NAIVE BAYES CLASSIFIER

O algoritmo Naive Bayes (NB) é um algoritmo de classificação supervisionado, probabilístico e baseado na Teoria de Conjuntos. É um dos principais utilizados em classificação por ser um dos mais eficazes e mais rápidos. Ele se baseia no teorema de Thomas Bayes (1702 - 1761), que calcula a probabilidade de um evento A, considerando evidências de B — a probabilidade condicional.

$$P(A|B) = \frac{P(B|A) \times P(A)}{P(B)}$$





NAIVE BAYES

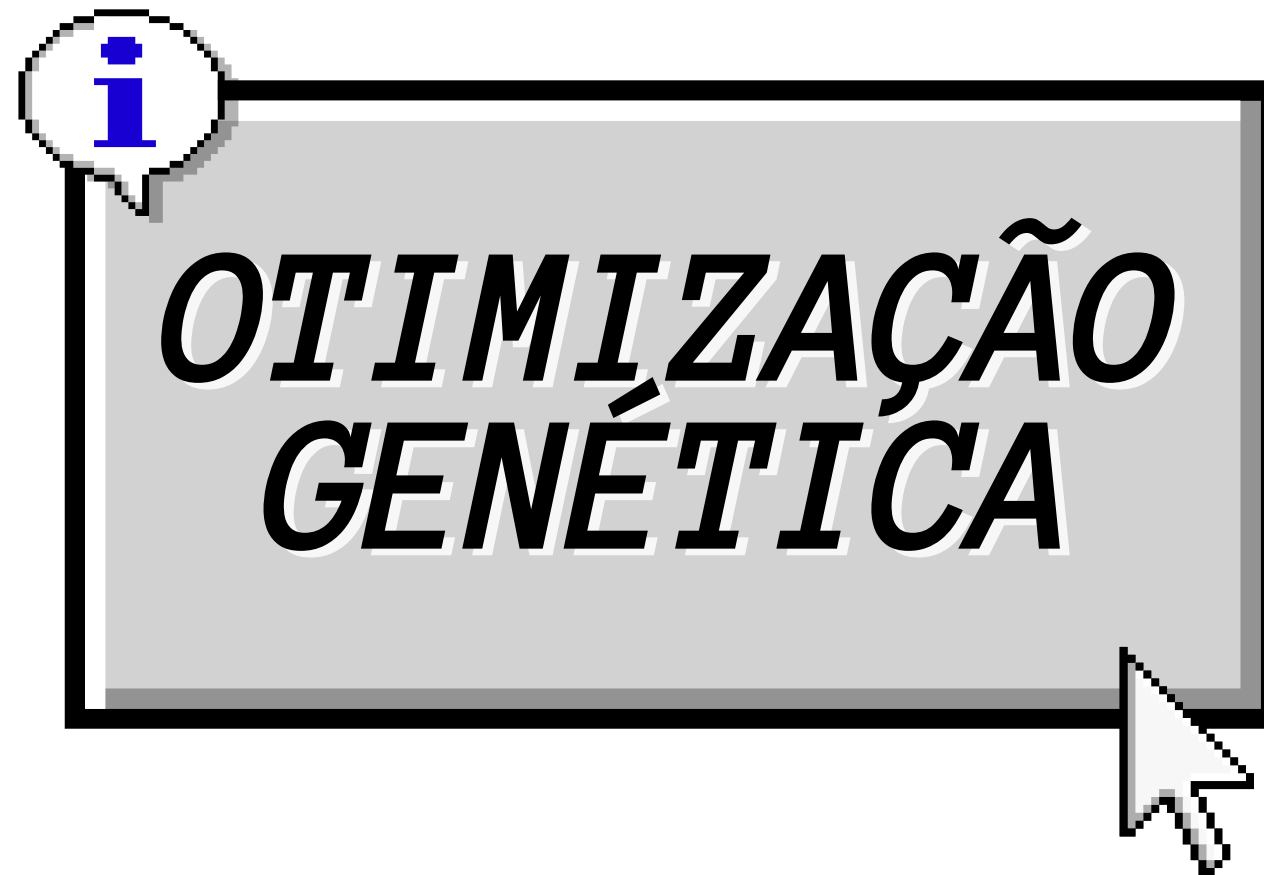
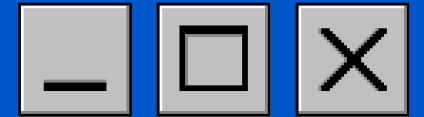
O nome Naive Bayes, além de Thomas Bayes, vem de ingênuo em inglês. O nome se dá pelo algoritmo tomar como premissa a independência condicional das características dos dados, o que é meio “ingênuo”, mas facilita os cálculos, pois, ao invés de calcular o teorema de Bayes, propriamente, ele simplifica a relação dos conjuntos por uma série de multiplicações.

$$\hat{y} = \arg \max_y P(y) \prod_{i=1}^n P(x_i | y),$$



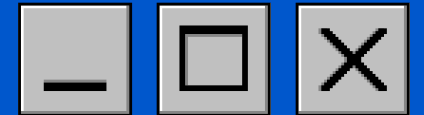
Thomas Bayes (1702 - 1761), fonte:
Times Literally Supplement





Algoritmos Genéticos (GA) são uma opção meta-heurística para otimização de algoritmos. Ou seja, utiliza princípios da busca heurística para encontrar os melhores parâmetros ou atributos para um determinado algoritmo.





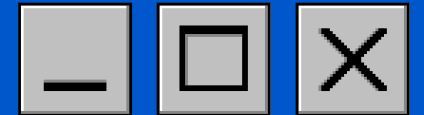
OTIMIZAÇÃO

Para NB, existem algumas formas de como um algoritmo genético pode entrar para otimizar a classificação:

- Seleção de features;
- Otimização de hiperparâmetros;
- Classificação híbrida.

Neste caso, utilizamos um algoritmo genético de seleção de features para otimização da classificação. Isso não altera o funcionamento do NB, apenas filtra quais vetores chegariam ao algoritmo.





OTIMIZAÇÃO

Considere o vocabulário abaixo:

[bom, ruim, ótimo, péssimo, muito, pouco]

O AG cria indivíduos como:

Indivíduo A: [1, 0, 1, 0, 1, 1]

Indivíduo B: [1, 1, 0, 0, 0, 1]

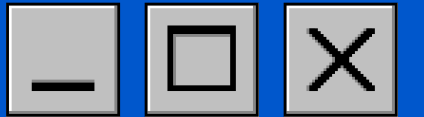
Indivíduo C: [0, 0, 1, 1, 0, 0]

Após os treinamentos:

- Indivíduo A → acurácia 84%
- Indivíduo B → acurácia 78%
- Indivíduo C → acurácia 61%

O AG então seleciona os melhores indivíduos, cruza os melhores atributos, cria uma nova geração de indivíduos e repete o processo. Depois de várias gerações, o GA evolui para o conjunto ideal de palavras de entrada para o NB.





[Home](#)

[Content](#)

[Contact](#)

OBRIGADO! :))

igor S. C.

Nina M. O.

Fernando T.

Guilherme F.

Kauã S.

[Github do projeto](#)

